Taming the Internet of Things with KNIME

Data Enrichement, Visualization, Time Series Analysis, and Optimization

Phil Winters Phil.Winters@knime.com
Aaron Hart Aaron Hart@knime.com
Rosaria Silipo Rosaria.Silipo@knime.com

Table of Contents

Summary

There is an explosion of sensor data becoming available, leading to the term Internet of Things. But how difficult is it to pull all that data together to use it to make more intelligent decisions?

In this paper, we pull 8 public sensory data sources, transform and enrich them with responses from external RESTful services mainly from the web in order to create profiles and segments around customers.

We then apply machine learning, time series analysis, geo-localization, and network visualization to take that data and make it actionable. In particular, we optimize the machine learning model size in terms of the smallest number of required input features, and the parameter values of the time series auto-regression model.

A few different techniques have been employed in visualization: geo-localization by means of the KNIME Open Street Map (OSM) integration; route localization using the ggplot R library; and network graph visualization with the KNIME Network Analytics extension. Each of these visualization techniques shows a different aspect of the data and of the KNIME open architecture, which makes integration of data and tools very easy.

Data and workflows are available for downloads at http://www.knime.com/white-papers#IoT , while the KNIME open source platform is downloadable from the KNIME site at http://www.knime.org/knime-analytics-platform-sdk-download .

The Internet of Things

There has been a lot of talk about the Internet of Things lately, especially since the purchase by Google of Nest, officially opening the run towards the intelligent household system (http://www.wired.com/2014/01/googles-3-billion-nest-buy-finally-make-internet-things-real-us/).

Intelligent means controllable from a remote location and capable of learning the inhabitants' habits and preferences. Companies working in this field have multiplied over the last few years and some of them have been acquired by bigger companies, like SmartThings by Samsung for example (http://recode.net/2014/08/14/internet-of-bling-samsung-buys-smartthings-for-200-million/).

However it is not only households that can be intelligently interconnected, cities – i.e. smart cities – represent another area for the application of the Internet of Things. One of the first smart cities around the world has been Santander in the north of Spain (http://www.smartsantander.eu/). Sensors have been installed around the city to constantly monitor temperature, traffic, other weather conditions, and parking facilities.

The Internet of Things poses a great challenge for data analysts, on the one hand because of the very large amounts of data created over time and on the other because of the algorithms that make the sensor-equipped object (house, or city) capable of learning and therefore smarter. But to date, there have been no public examples combining data and analytics to share and learn from.

We at KNIME decided to change that and put KNIME to work to add some more intelligence to one of the many Internet of Things applications available around the world.

The Use Case: Capital Bikeshare in Washington DC

As an example of an Internet of Things application, we need a use case that is easy for everybody to understand and with publicly available data to reproduce. After some search, we found a bike share system in Washington DC called Capital Bikeshare (http://www.capitalbikeshare.com/).

Capital Bikeshare

Quoting from Capital Bikeshare home page (Fig. 1):

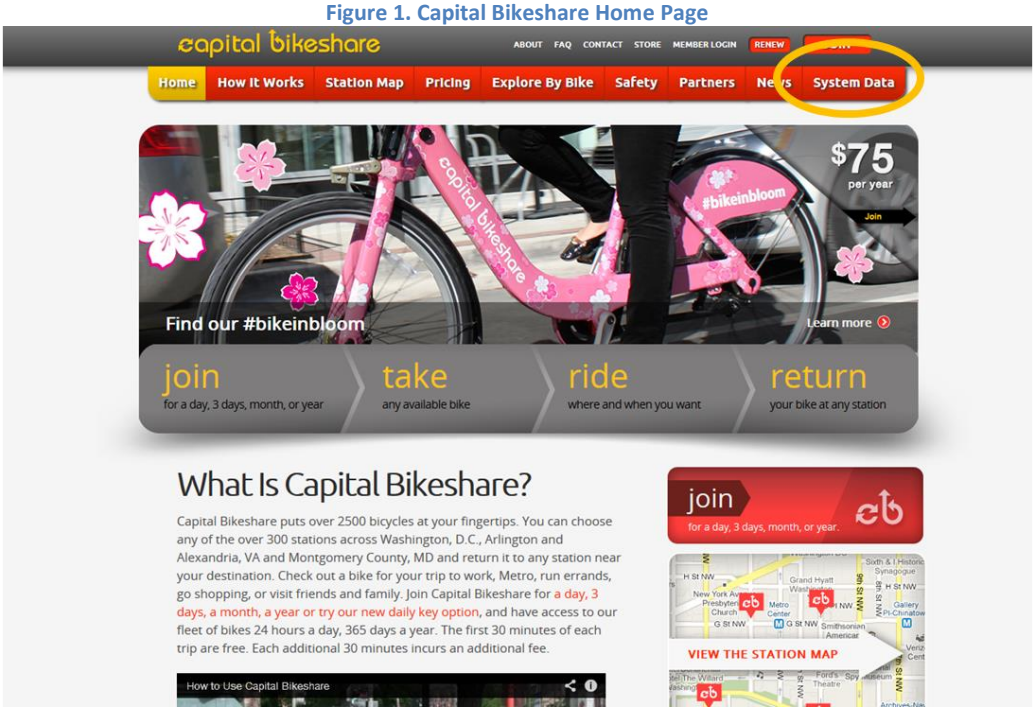
"*Capital Bikeshare puts over 2500 bicycles at your fingertips. You can choose any of the over 300 stations across Washington, D.C., Arlington and Alexandria, VA and Montgomery County, MD and return it to any station near your destination. Check out a bike for your trip to work, Metro, run errands, go shopping, or visit friends and family. Join Capital Bikeshare for a day, 3 days, a month, a year or try our new daily key option, and have access to our fleet of bikes 24 hours a day, 365 days a year. The first 30 minutes of each trip are free. Each additional 30 minutes incurs an additional fee*."

Capital Bikeshare is a bike service for tourists and residents to move around the Washington DC downtown area, therefore reducing traffic and pollution and promoting a healthier lifestyle. Notice the 30 minutes rule: the first 30 minutes are free and after that each additional 30 minutes incurs a small fee.

Each bike carries a sensor, which sends the current real-time bike location to a central repository. Capital Bikeshare makes all its historical – of course anonymized – data from each bike available for download on its web site, as you can see from the "System Data" tab at the upper right corner in its home page (Fig. 1). These public data have been downloaded for use in this study.

The downloaded data provide the space-time points for each bike. As all kinds of information can now be found on the web, it was then easy to integrate that with topology, elevation, local weather, holiday schedule, traffic situation, the location of businesses, touristic attractions, and other types of information widely available on the Internet via web or REST services.

Figure 1. Capital Bikeshare Home Page



The Town

Washington DC, like many other big cities, is overcrowded with cars and people. With a population of more than half a million residents and as many commuters from the surrounding subsurbs of Maryland and Virginia during the week as well as almost as many tourists pouring in during holidays and weekends, it is easy to imagine how driving a car, especially in downtown areas, can become a complex experience.

A bike ride represents a multi-win solution: moving easier and faster throughout the town, reducing stress, reaching places otherwise not reachable by car, and producing an enjoyable experience especially if the weather cooperates.

Bikeshare customers consist of two groups.

- The registered customers, usually owning a month/year card, who are typcially residents and work commuters.
- The casual customers, mainly tourists, with numbers peaking during nice weather days and holidays.

The Business Challenge

The main challenge of a bike share business is the timely restocking of bike stations. It is absolutely counterproductive for business to have customers arrive at an empty bike station – when starting the trip, or to an overfilled bike station – when returning the bike. Both situations delay operations and create a bad customer experience, especially if the goal is to take advantage of the 30 minutes rule.

An empty or an overfilled station damages not only the image of the business, but also the business directly in terms of money. Indeed, an agreement between Capital Bikes and the city of Washington DC stipulates that any station without bikes or any station with no free slots for more than one hour represents a violation and incurs a financial penalty. This puts even greater pressure on the bike management team to restock the bike stations in time.

Currently the monitoring of the bike station is human-driven; i.e. somebody is allocated to watch the bike station situation and who decides when it is time to shuffle bikes around replenishing empty stations with bikes from overfilled stations. The decision about when the number of bikes is excessive or insufficient for a given station is based solely on gut feeling and experience. Optimizing the timing of this decision would lead to a minimal number of reshuffling actions and would prevent the overfilling or complete emptying of all bike stations.

The Data

The .csv files, downloadable from the Capital Bikeshare web site http://www.capitalbikeshare.com/trip-history-data, cover a quarter year each and contain basic information about each bike trip, such as:

- **Duration** – Whole duration of the bike trip
- **Start date** – Start date and time
- **End date** – End date and time
- **Start station** – Starting station name and number
- **End station** – Ending station name and number
- **Bike #** - ID number of bike used for the trip
- **Member Type** – This field lists whether user was a registered (annual or monthly) or casual (1 to 5 day) member.

All private data, including member names, have been removed from all files.

Taming the IoT: an Overview of the Approach

The goal of this project is to define an optimal alarm system to reshuffle bikes across stations. The starting data has been provided by the Capital Bikeshare web site. This data contains only bike trip related information. No other information about itinerary, weather, traffic conditions, or calendar information is included. Such information, however, is widely available on the web for many geographical regions.

The work was divided in three subsequent steps, as for most data analytics projects: Data Preparation, Visual Investigation, and Predictive Analytics.

Data Preparation
In the DataPreparation workflow, after reading the data files in, we extend the original data set with weather, topology, geography, holidays, and traffic related information acquired from independent sources available on the web.

After collecting a general enough data set, we expand the data in their best space, adding a number of descriptive measures and aggregations.

Visual Investigation
Data visualization follows the data preparation phase. This is always an important step preceding the analytics phase. Indeed, it is useful to visualize the data to check for obvious data clusters, interesting outliers, or statistical properties before proceeding with the application of analytics techniques.

In the "Visual Investigation" workflow, the number of available bikes in all bike stations across the whole Washington DC is visualized with the help of the KNIME Open Street Map Integration.

In a next step, we visualize the number of bikes covering the routes connecting bike stations. This part has been implemented with the help of a Google API to retrieve routes and of the R graphical library ggplot to depict them.

And finally, we draw on network analytics to picture bike routes and extract those geographical areas most frequented by the bikers in our data set.

Predictive Analytics
Now that we have an idea of how the data set is organized, the project can proceed with the analytics phase. The main goal here is to build an optimal automatic alert system. The system must indicate the right time to restock a station with bikes, which should be neither too early nor too late. Not too early because we want to minimize the number of bike reshuffles, not too late because we do not want to incur a fine. To implement this system we try to anticipate human behavior: the alert must come one hour before the human monitor would start the restocking.

We also implemented a prediction system to predict the total number of registered and casual customers in the city for the next hour. This part was implemented using the new Time Series metanodes in KNIME 2.10 and followed the time series prediction steps described in the KNIME whitepaper "Big Data, Smart Energy, and Predictive Analytics".

Data Preparation: Reading, Enriching, Transforming

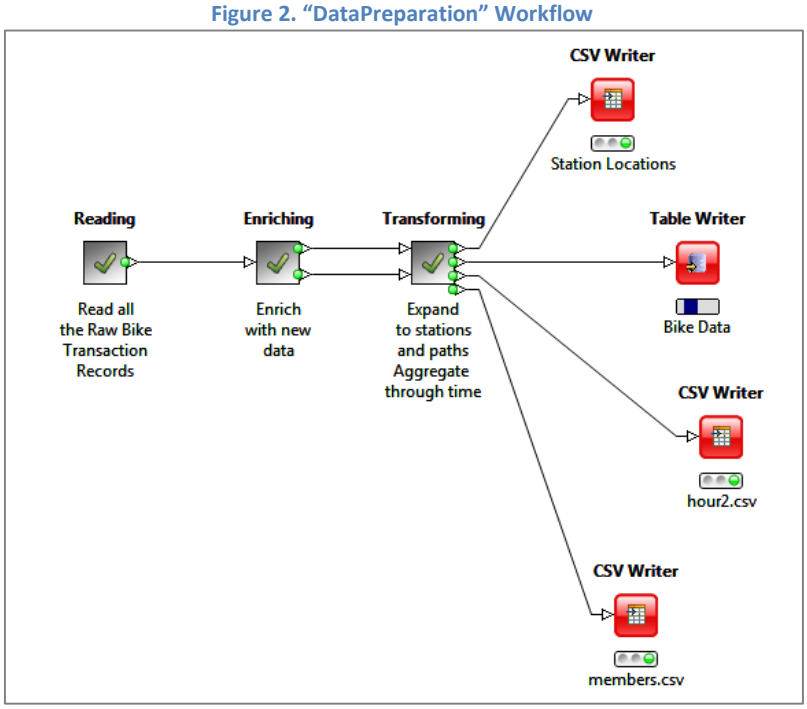
The "DataPreparation" workflow implements all data preparation tasks, including:

- reading the data with their different formats,
- extending the original data set with information downloaded from the web,

- transforming the data structure for a more meaningful representation of the predictive analytics workflow.

Final results are then saved into three different files. The final workflow is reported in Figure 2, with a metanode implementing each one of the three tasks respectively.

Figure 2. "DataPreparation" Workflow

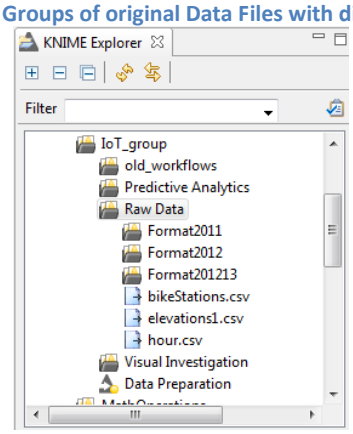


Reading the Sensor Data

The first metanode of the DataPreparation workflow is named "Reading" and reads the bike sensor data, as downloaded from the BikeSharing web site http://www.capitalbikeshare.com/trip-history-data.

As with many sensor data sources, the data format has changed over the years. In this workflow, three loops loop over data files with consistent formats, clean and standardize the resulting data sets, and collect all the final data together in one single master data table. All columns containing a date are also converted to DateTime objects with the String to Date/Time node.

Figure 3. Three Groups of original Data Files with different Formats



All original files are grouped under the workflow group “IoT_group/Raw Data”. As you can see in the KNIME Explorer panel, the data files are grouped together by format in folder “Raw Data”. That is, all data files with a format consistent with the format used in 2011 have been grouped together under folder “Format2011”, and so on (Fig. 3).

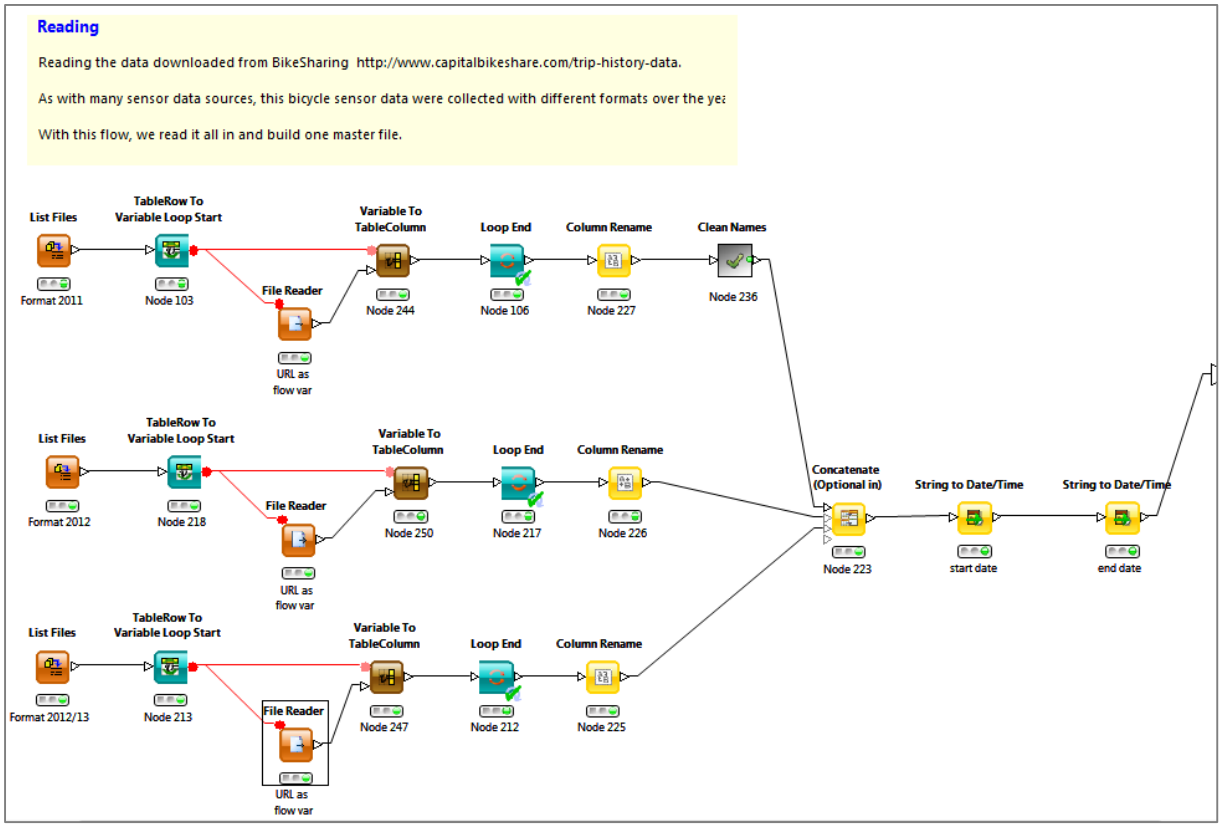
To read all files in a folder, the “List Files” node followed by a reading loop seems to be the most appropriate solution.

The “List Files” node accepts a folder path in the configuration settings and produces the absolute paths and URLs of all files contained in that folder. The reading loop then goes through every single file path produced by the “List Files” node, feeds this path as a flow variable to a “File Reader” node, and reads its content. The “Loop End” node finally collects all data output by the “File Reader” node at each iteration, i.e. for each file path.

Notice the “Variable To Table Column” node after each “File Reader” node. This appends the name of the current file to each data row and thus keeping track of each data row’s original file.

After a few cleaning and standardization operations with the “Column Rename” node and with the “Clean Names” metanode, the workflow concatenates the three resulting data sets and converts all date and time values into DateTime objects.

Figure 4. Sub-flow in the Reading Metanode of the DataPreparation Workflow



Enrich with dditional data from files and REST services

The imported data table is organized by bike trip. Each data row contains the bike ID, the start station (and start terminal ID), the end station (and end terminal ID), the member type (registered or casual), and the trip duration. There is nothing about geography, weather, or other context related variables.

In the "Enriching" metanode, we add latitude, longitude, and elevation to each bike station and infos about weather in Washington DC during that bike trip. All those values (latitude, longitude, elevation, and weather) can be retrieved from the Google API REST services, for example with request:

http://maps.googleapis.com/maps/api/elevation/json\?locations=<latitude value>, <longitude value>&sensor=false

as well as from files available on the UCI Repository (https://archive.ics.uci.edu/ml/datasets.html).

In the "DataPreparation" workflow associated with this whitepaper, we mainly used files from the UCI Repository. This is because the workflow must be able to run even if there is noconnection to the Internet. However, the "Elevation" metanode, inside the Enriching metanode, offers an example of how to build a request to a RESTful service, in this case to a Google API RESTful service.

Accessing a REST Service

The sub-workflow that submits the requests to the Google API RESTful service is shown in Figure 5. This sub-workflow consists of two parts: a loop posting a *request* and collecting the response for each data row and a metanode, named "extract", also containing a loop to *interpret all received responses* (Fig. 6).

Figure 5. The sub-workflow accessing the Google REST service and interpreting the response

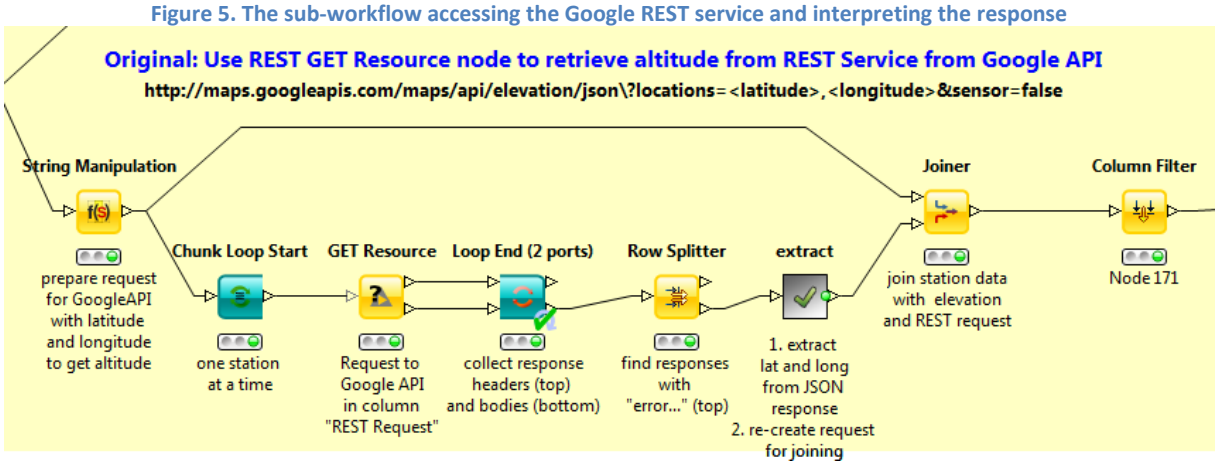
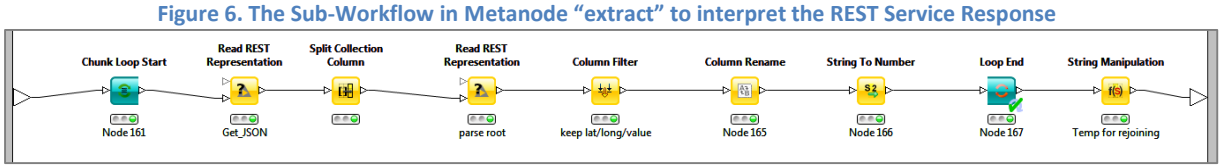


Figure 6. The Sub-Workflow in Metanode "extract" to interpret the REST Service Response



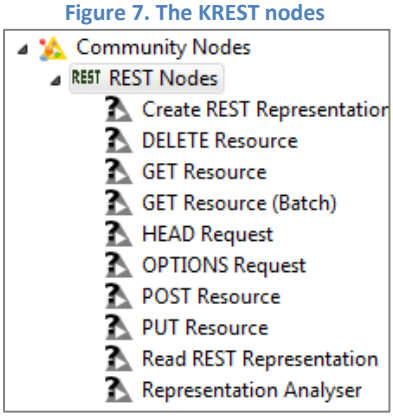
The center piece of the request loop is the "GET Resource" node; the center piece of the loop in the "extract" metanode is the "Read REST Representation" node. Both nodes are part of the KREST (REST nodes for KNIME) trusted extension (http://tech.knime.org/book/krest-rest-nodes-for-knime-trusted-extension). The KREST nodes can be downloaded like any other KNIME extension using the File -> Install KNIME Extensions … option, selecting "KNIME Community Contributions – Other" and then "REST Client", and following the extension installation wizard.

The KREST node plugin provides nodes that allow KNIME to interact with RESTful webservices (Fig. 7). There are two types of nodes:

- *Submitters* take over the actual communication with the REST server
- *Helpers* convert REST resource representations into KNIME tables and vice versa.

All REST communications can be logged in a separate log file (to be activated in the KNIME preferences page).

Figure 7. The KREST nodes

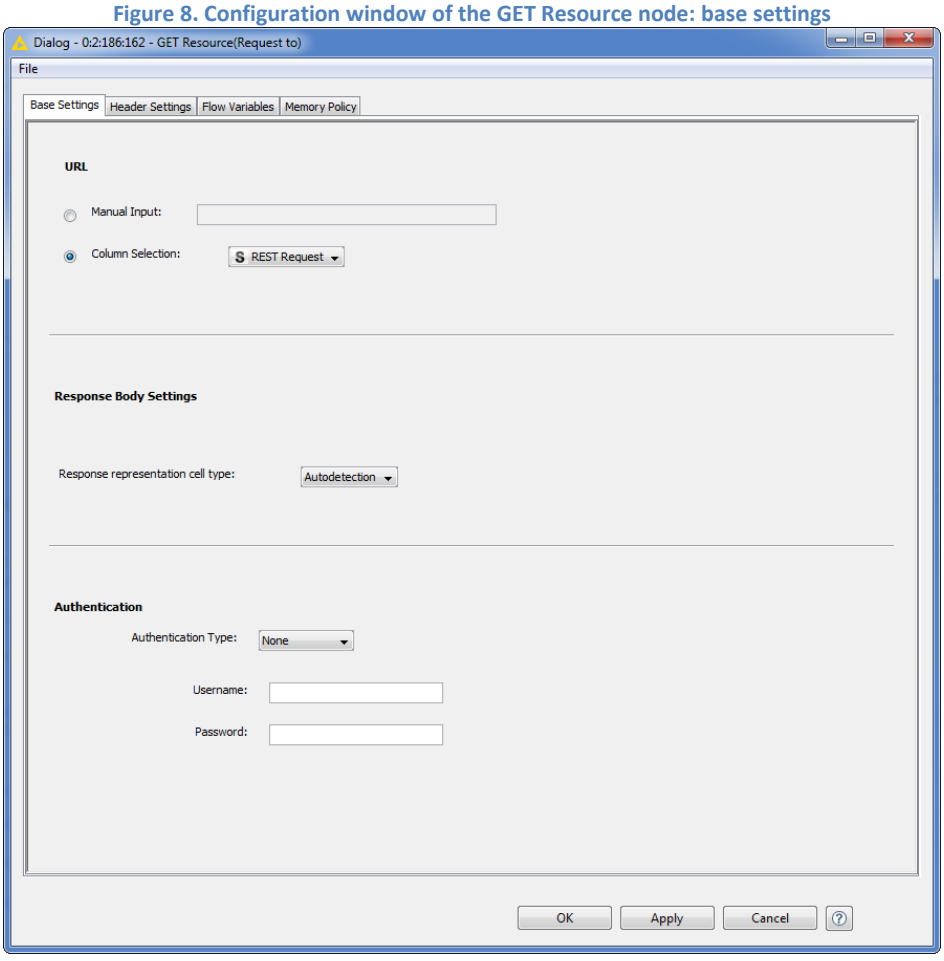


In the sub-workflow in Figure 6, we use a "String Manipulation" node to build the request string for the Google elevation REST service and apply the join command with latitude (lat) and longitude ($long$) values from each data row, as:

```
join("http://maps.googleapis.com/maps/api/elevation/json\?locations=",strin
g($lat$),",",string($long$),"\&sensor=false")
```

This produces a data table with bike station name, station ID, latitude, longitude, and the request string to obtain the elevation from the latitude/longitude coordinates.

Figure 8. Configuration window of the GET Resource node: base settings



Then we use a “GET Resource” node to submit the HTTP GET request strings produced by the “String Manipulation” node.

In the configuration window of the “GET Resource” node (Fig. 8), you can define the REST request in the “Base Settings” tab either manually or automatically using the first value of an input data column. In the “Base Settings” tab you can also provide authorization credentials, if needed, and the response representation format as pure text (String) or XML. The response representation format can also be autodetected by the node itself, after selecting the “Autodetection” option.

The “Header Settings” tab in the configuration window of the “GET Resource” node allows for customization of the request header.

However, the “GET Resource” node only processes one REST request at a time, which means it needs an input data table with only one row at a time. In order to process all REST requests for all bike stations, we use a chunk loop (“Chunk Loop Start” node, “Loop End (2 ports)” node) processing one data row at a time.

The REST response is presented in two output data tables: one shows the header (including the status code) and one shows the unprocessed response body from the REST service. The “Loop End (2 ports)” node collects both output data tables even though we are only interested in parsing the unprocessed response of the REST service.

The communication between the REST service and the “GET Resource” node can be logged in a separate log file. The logger is activated and adjusted in the KNIME Preferences page under REST Nodes. More information and a tutorial on how to use the KREST nodes can be found at http://tech.knime.org/book/krest-rest-nodes-for-knime.

Again using a chunk loop, the “extract” metanode extracts the elevation value from each REST response. First a “Read REST Representation” node extracts the values string from the JSON structured response. Then, a second “Read REST Representation” node extracts the values of latitude, longitude, elevation, and resolution from the JSON subclass.

The “Read REST Representation” node converts a REST resource representation into a KNIME data table. In the configuration window, the representation data format can be selected manually from a menu or extracted from the response header at the top input port. The default values for all settings can be adjusted again in the Preferences page under KNIME -> REST Nodes.

Calculate Line of Site Distance based on Latitude and Longitude

After acquiring latitude, longitude, and elevation of the starting and ending point of the bike path, we can also add the elevation change and the line of site distance covered for each bike trip. The elevation change is just the difference between the two elevation numbers at the beginning and at the end of the bike trip. The line of site distance covered can be calculated from the (latitude, longitude) coordinates of the starting and ending point by means of the following formula:

```
% Earth radius in km
   R = 6371;
% Coordinates of two points in latitude and longitude
   (lat1, long1)  and (lat2, Long2)

% Converts degrees into gradians
   lat1 = lat1*2*pi/360;
   lat2 = lat2*2*pi/360;
   long1 = long1*2*pi/360;
```

```
    long2 = long2*2*pi/360;

% calculate latitude/longitude difference
    dlat = lat2-lat1;
    dlong = long2-long1;

%apply formula to calcuate distance(A,B)
    a = (sin(dlat/2))^2 + cos(lat1)*cos(lat2)*(sin(dlong/2))^2;
    c = 2*atan2(sqrt(a), sqrt(1-a));
    d = R*c*1000;       % d in km, mulptiplied by 1000 to get the distance m
```

Transforming a List of Bike Trips into a Time Series of Aggregated Values

Now that we have integrated additional information from the web into the original data set, we would like to move from a pure list of bike trips to a more informative description with the number of available docks in each bike station, the number of bike renters, and the number of bikes on a given path.

Number of Bikes Available at each Station Over Time

The metanode "Delta" inside the metanode "Expand" goes through the list of bike paths and extracts the "Start Station" with its "Start Date/Time" and the "End Station" with its "End Date/Time". Count = -1 is assigned to the "Start station" at the "Start Time" hour, i.e. bike is leaving the station, and Count = +1 is assigned to the "End Station" at the "End Time" hour, i.e. bike is reaching the station.

Metanode "stations" starts from the number of docks and bikes available at time 0 at each station and calculates the net sum of bikes coming and going at each hour at each station ("Count cumulative sum"). "Count cumulative sum" must also take into account the periodic bike restocking, generating the "Adjusted Cumulative Sum" data column.

Number of Bike Renters over Time

The "Members" metanode aggregates the "Delta" output data in terms of registered and casual users. To do that, it uses a "One2Many" node to map the "registered"/"casual" values in the "Member type" column into 1/0 values in two data columns, named "registered" and "casual". Results are then aggregated through a "GroupBy" node across date and time. The time series of registered and casual bike renters by the hour is produced by the output port.

Total Number of Bikes on a Given Bike Path

Finally, the metanode "A-B Paths" produces the total number of bikes for each path for each date and hour.

The output data, including a copy of the original data fed into the "Delta" metanode, are written into files located in the "Raw Data" folder.

Note. "DataPreparation" workflow contains just a few examples of how you can import, enrich, and transform your data. We used a number of other transformations to produce the data files used in subsequent workflows; not all of them are included in the "DataPreparation" workflow.

Visual Investigation: Scatter Plots, Maps, and Network Graphs

Before starting with the predictive analytics part of this project, we walked through some visual investigation to get a better idea of the data we are dealing with.

The first task in visualization was to geographically localize all the bike stations on a Washington DC map. This could be done in many ways: through R code or using the KNIME Open Street Map (OSM) Integration available as a KNIME extension under KNIME Labs. We went for this second option.

The second step would definitely involve a study of the traffic flow, in terms of which stations are most targeted and which bike routes are most covered.

To describe the bike activity at each station, the number of bikes reaching the station and the number of bikes leaving the station were calculated for the whole time window. The net difference between these two numbers characterizes the station as a sink (more bikes arriving than leaving), a source (more bikes leaving than arriving), or as a neutral station (same number of bikes leaving and arriving at the station). We can then perform our visualization on the same OSM map as before and showing all stations as sinks or sources, to get an idea of where people are mainly leaving from (bike stations close to subway stations, touristic starting points, …) and where people are mainly heading (touristic destinations, lively areas of town, etc…).

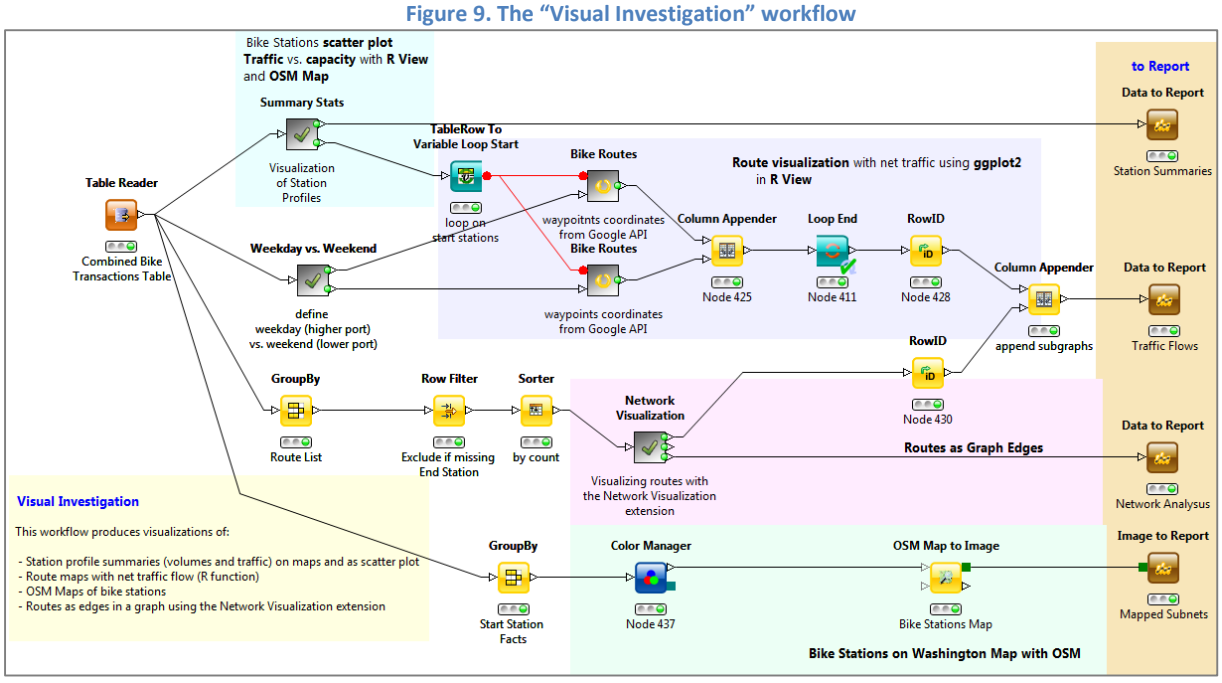
We subsequently know which station is the main source and which one is the main sink of bike rides. It would be interesting to visualize the routes connecting all bike stations and the number of bike rides they support. The KNIME Open Street Map Integration does not provide bike route visualization. But Google Maps does. We turned then to another Google RESTful service to download all waypoints and their coordinates of the bike routes connecting bike stations. The ggplot2 library in R visualizes the route coordinates with the information of their bike traffic.

Finally, we can also represent the bike traffic connecting one station to another in the shape of a network graph, using the Network Visualizer node of the KNIME Network Analytics extension. An edge in the graph represents a bike route in the city. The edge size represents the amount of bike traffic on the corresponding route.

The workflow used for these four visualization tasks is called “Visual Investigation” (Fig. 9) and consists of 4 parts:

- visualizing the *bike stations on a Washington DC map* using the KNIME OSM integration;
- representing stations as sinks or sources either on a *scatter plot* or on a OSM map;
- showing the *bike route map* with the R ggplot2 library;
- visualizing the bike station network together with its traffic information as a *network graph*.

Figure 9. The “Visual Investigation” workflow



The workflow starts with reading the file "Combined Bike Transactions.table" created by the "DataPreparation" workflow. This file contains all bike trips, with start and end station name, ID, longitude, and latitude, start and end time, duration, bike ID, and distance covered.

Pinning Bike Stations in Washington DC with Open Street Map Integration

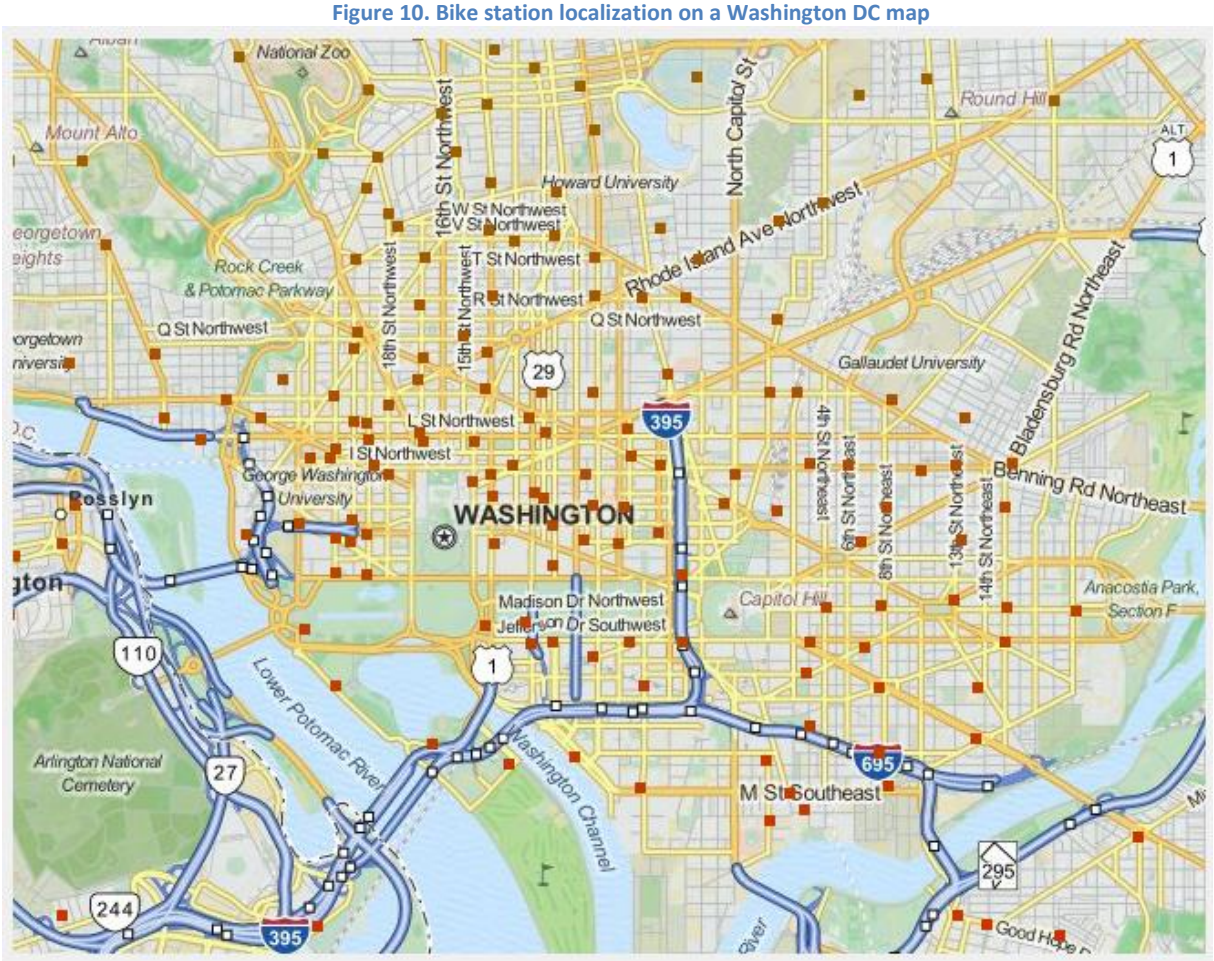
The first visualization task is to visualize all bike stations on a Washington DC map. In this case we need a list of stations with their latitude and longitude coordinates and an "OSM Map to Image" node.

We collect all start stations with their latitude and longitude from the input file using a "GroupBy" node. The "GroupBy" node groups by station name and exports the average values of latitude and longitude. Since the latitude/longitude coordinates always have the same values for the same station, the same output data table could be obtained taking the "First" value for each group in the "Options" tab of the "GroupBy" node's configuration window.

The average latitude/longitude values then feed an "OSM Map to Image" node from the OSM Integration extension, producing a Washington DC map with points for the bike stations (Fig. 10).

The lowest branch of the "Visual Investigation" workflow depicted in pale green in Figure 9 is the part of the workflow that localizes the bike stations on a Washington DC map.

Figure 10. Bike station localization on a Washington DC map



Station Bike Traffic

We now want to refine the map above with information about bike sources and bike sinks. This representation can help us to organize periodic bike restockings.

First of all we count the number of rows with a given starting station, i.e. the number of bikes leaving that station. Similarly, we count the number of rows with a given ending station, i.e. the number of bikes reaching that station. If we now join the two data tables on the station name, we have the same station working as a starting point and as an end point, with the number of bikes leaving ("Bike #" column) and the number of bikes reaching ("Bike # (1)" column) the same station spanning the whole time window.

- The sum of the number of bikes leaving and reaching the station ("volume") gives a measure of the overall traffic at the station.
- The difference between the number of bikes reaching the station and the number of bikes leaving the station ("net") describes the net lack or excess of bikes for this station.
- The absolute value of the net column ("abs_net") describes how urgently the stations need to be restocked with bikes.
- Finally, the ratio between the absolute value of the net number of bikes (abs_net) and the volume of the station ("churn" = abs(net)/$volume$*100) measures the discrepancy between the bike excess/lack and the station volume.

The difference between the number of bikes reaching the station and the number of bikes leaving the station ("net") is used to describe a station as a sink (collecting bikes, "net" > 0) or a source (distributing bikes, "net" < 0). A "Rule Engine" node attributes the label "Sink" or "Source" to each station depending on the value of the data column named net.

We want to produce a scatter plot depicting absolute net traffic ("abs_net") vs. station volume ("volume"). We could use the "Scatter Plot" node from the KNIME "Data Views" category. Or we could use the R scatter plot function via the ggplot2 library, using the following R code in an "R View (Table)" node:

```
require(ggplot2)

x = knime.in$"volume"
y = knime.in$"abs_net"

Size = knime.in$"abs_net"
Type = knime.in$"type"

points = geom_point(aes(x, y,
                        color=Type,
                        shape = Type,
                        size = Size),
                        knime.in)

clean_theme = theme(panel.background = element_blank(),
             plot.title = element_text(size=20, face="bold", colour = "black"),
             panel.border = element_rect(color = "black", linetype = "solid",
             fill = "transparent"),
             axis.title.x = element_text(size=14, face="italic", colour = "black"),
             axis.title = element_text(size=14, face="italic", colour = "black"),
             axis.text = element_text(size=12, face="italic", colour = "black"),
             legend.text = element_text(size=12, face="italic", colour = "black"),
             panel.grid = element_blank()
             )

colors = scale_colour_manual(values= c("red", "blue"))
shapes = scale_shape_manual(values=c(6,2))
size = scale_size(range=c(1,9), guide=FALSE)
labels = labs(title="Station Profiles", x = "Station Volume" , y ="Absolute Net
Traffic")

ggplot() + points  + labels + colors + shapes + size + clean_theme
```


knime.in$"type" contains the "Sink"/"Source" labels and defines the color and shape property, while knime.in$"abs_net" is used to shape the size of the plot points. The resulting scatter plot is shown in figure 11.

We can also just replace the station points on the OSM map in Figure 10 with points shaped as sink (red reverse triangles) or sources (blue triangles) with size proportional to the abs_net values. "Sinks" in column "type" are associated with the color red and "Sources" with the color blue by a "Color Manager" node. There are "Sinks" and "Sources" of different entities depending on the number of bikes collected or distributed, as described by the data column named "abs_net". The "Size Manager" node then changes the dot size according to the "abs_net" value. Finally, a "Shape Manager" node associates a triangle with all "Source" stations and a reverse triangle with all "Sink" stations. The resulting data table is re-fed into an "OSM Map to Image" node, producing the Washington DC map in Figure 12.

Figure 11. Scatter plot of absolute net traffic vs. station volume using the ggplot2 R library

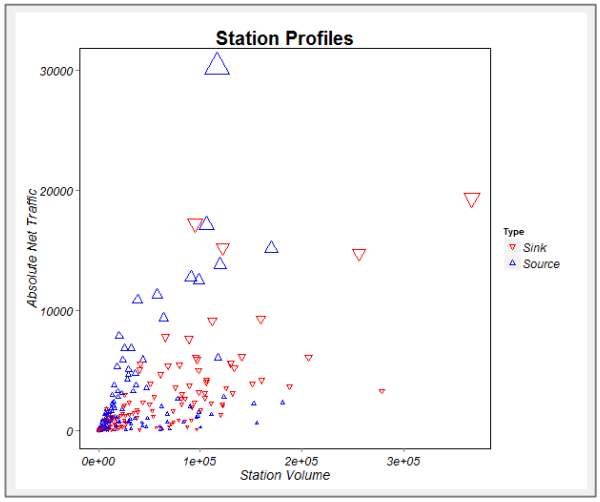
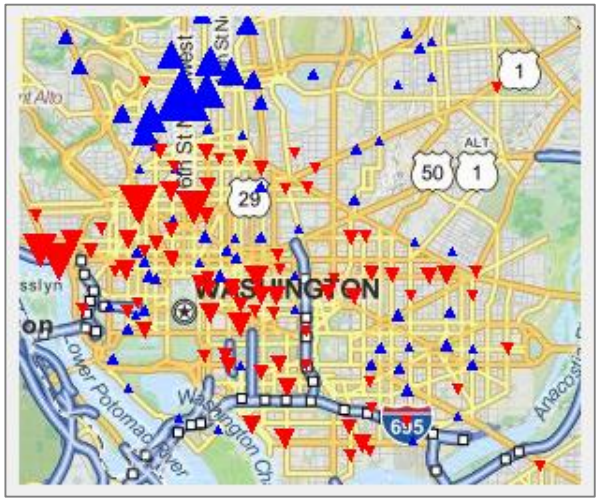
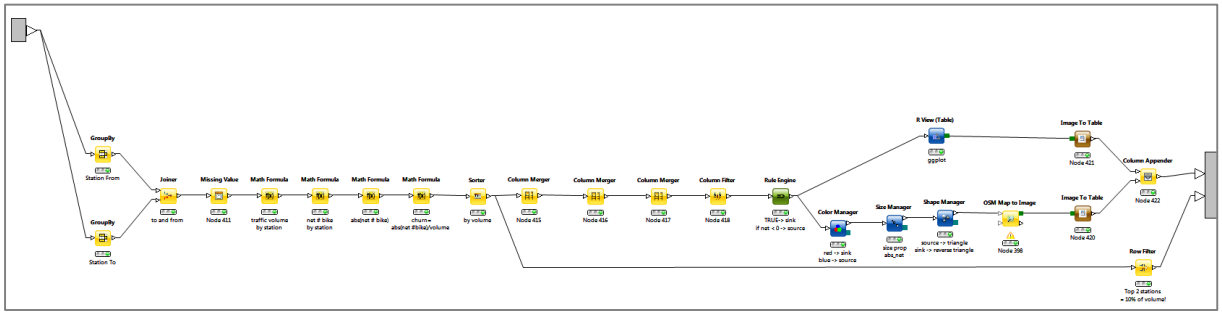


Figure 12. Map of bike stations in Washington DC (blue = sources, red = sinks)



Both branches, the one with the R code and the one with the OSM node are contained in the "Summary Stats" metanode (Fig. 13) in the upper left corner of the "Visual Investigation" workflow in figure 9.

Figure 13. Sub-workflow of the "Summary Stats" metanode to produce the scatter plot in Figure 11 and the bike station map in Figure 12



Traffic Flow on Bike Routes

Figures 11 and 12 show the traffic situation at different bike stations. We would now like to visualize the traffic not only at the stations, but also on the bike routes around the city. There are many ways to visualize route segments, for example as a basic statistic from the most crowded segment to the least crowded segment; as a road visualization on a city map; a representation of sinks and sources through an oriented graph via the network visualization node; and many more!

In this section, we visualize the bike routes on a city map with thicker lines for the most crowded routes. For each route we have the coordinates (latitude, longitude) of the start and end stations. We need the coordinates of each point (waypoint) of the route inbetween. As usual, the coordinates of such waypoints can be retrieved through a Google API REST service with the service call:

```
http://maps.googleapis.com/maps/api/directions/json?
origin=<latitude>,<longitude>&destination=<latitude>,<longitude>
&sensor=false&avoid=highways&mode=bicycling
```

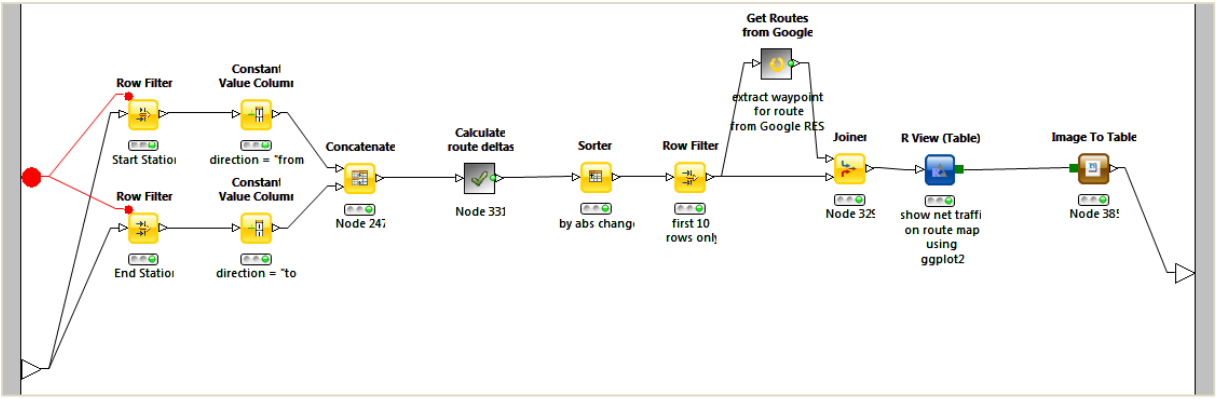
Again, using the "GET Resource" node and the "Read REST Representation" node, we query the Google API service and interpret the responses for each pair of origin and destination bike stations. This is contained in the metanode "GET Routes from Google" inside the "Bike Routes" metanode in the branch with a gray background in the upper part of the "Visual Investigation" workflow, depicted in Figure 9.

The lower output port of the "Summary Stats" metanode produces the list of start and end stations of the most trafficked bike routes. A "TableRow To Variable Loop Start" node then loops on all data rows and feeds the "Bike Routes" metanode (Fig. 14) as the loop body. This loop performs the following tasks:

1. Query the Google API to get the coordinates of the waypoints for all bike routes
2. Assess the number of bikes covering those routes for the whole screened time window
3. Feed an "R View" node with the following R code, using the ggplot() function from the ggplot2 library

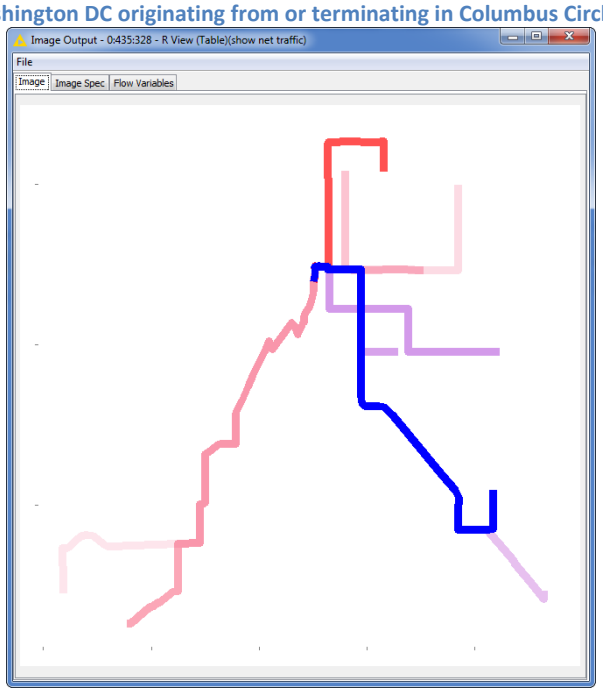
```
require(ggplot2)
require(grid)
clean_theme = theme(text = element_blank(),
                    panel.background = element_blank(),
                    panel.border = element_blank(),
                    axis.title.x = element_blank(),
                    axis.title = element_text(size=14, face="italic",
                    colour = "black"),
                    axis.text = element_text(size=12, face="italic",
                    colour = "black"),
                    legend.text = element_blank(),
                    legend.key = element_blank(),
                    legend.position = "none",
                    panel.grid = element_blank()
                    )

paths = geom_path(aes(
                      knime.in$"Long",
                      knime.in$"Lat",
                      size = 2,
                      group = knime.in$"route",
                      alpha = knime.in$"abs change",
                      color = knime.in$"net change"),
                  knime.in
                  )

colors = scale_colour_gradient(low="red", high="blue")
ggplot() + paths + clean_theme + colors
```

Figure 14. Sub-workflow in the "Bike Routes" metanode assessing the number of bikes on each bike route and retrieving the route waypoints from the Google API REST service



The ggplot function in the "R View" node produces a map image of the route segments represented through their waypoints. An "Image To Table" node imports that image for each bike route into a data table.

Figure 15. Bike routes in Washington DC originating from or terminating in Columbus Circle/Union Station bike station



Traffic Flow on Routes as Edges of a Network Graph

A second way to visualize traffic on bike routes is to represent each route as an edge of a network graph. To perform this kind of visualization, we need to work with the KNIME Network extension. After installing the KNIME Network extension, in the "KNIME Labs"/"Network" category we find a number of nodes to create, import, build, and modify a network graph (Fig. 16).

First, we delimit the number of routes to visualize to 250 only. This is to leave the network graph not so overcrowded and still readable.

The creation of a new network always starts with a "Network Creator" node that produces a new empty network object at the output port.

The "Network Creator" node is usually followed by an "Object Inserter" node, to insert objects from a data table as nodes and edges into the network. The input data table must provide pairs of nodes as (start node, end node) or the list of nodes associated with each edge of the network. We of course use

the (“Start Station”, “End Station”) pair to shape our empty network. The output is an updated network with bike stations as nodes and bike routes as connections between nodes.

Figure 16. The “Network” category under KNIME Labs

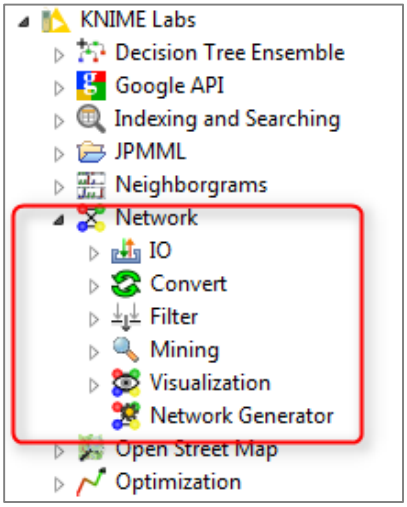
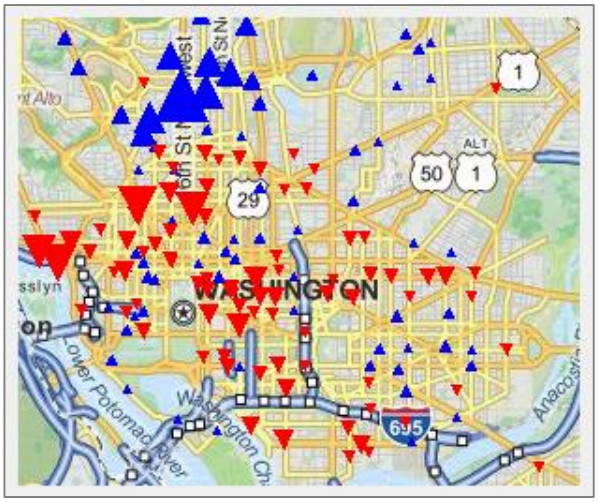


Figure 17. Map of bike stations in Washington DC (blue = “Sources”, red = “Sinks”)



The “Feature Inserter” node then follows the “Object Inserter” node. The “Feature Inserter” node inserts features from a data table into the network graph, node, or edge. The bike count for each route is converted into a heatmap with the “Color Manager” node, and into a size feature. We use a “Feature Inserter” node to insert both color and size features into the network graph at the edge level.

The “Subgraph Extractor” randomly extracts a limited number (100) of sub-graphs and places them into a data table. The top two sub-graphs only are selected and put back into a network object with the “Row To Network” node.

Finally the “Network Viewer” node produces an image of the selected two network subgraphs. The subgraph inherent to Columbus Circle/Union Station is shown in Figure 18 below.

One last remark, before closing this section about visualization techniques, refers to the configuration window of the “Network Viewer” node (Fig. 19). The “Network Viewer” node has to visualize the entire graph, taking visual properties for the whole graph (“General” and “Layout Settings” tabs), the nodes (“Node Layout” tab), and the edges (“Edge Layout” tab) into account.

The “Network Viewer” node shows the same image also via an interactive View window. The context menu of the interactive View window allows the user to hilite points and to display labels and tooltips at the node and edge level.

All the generated images have been sent to a series of “Data To Report” and “Image To Report” nodes to generate the report associated with this workflow.

Figure 18. Side by side comparison of traffic flow for balanced and skewed station sub-networks. Dupont Circle has a strong inflow of traffic from Adams Mill & Columbia Rd while Union Station shows balanced traffic flows in both directions

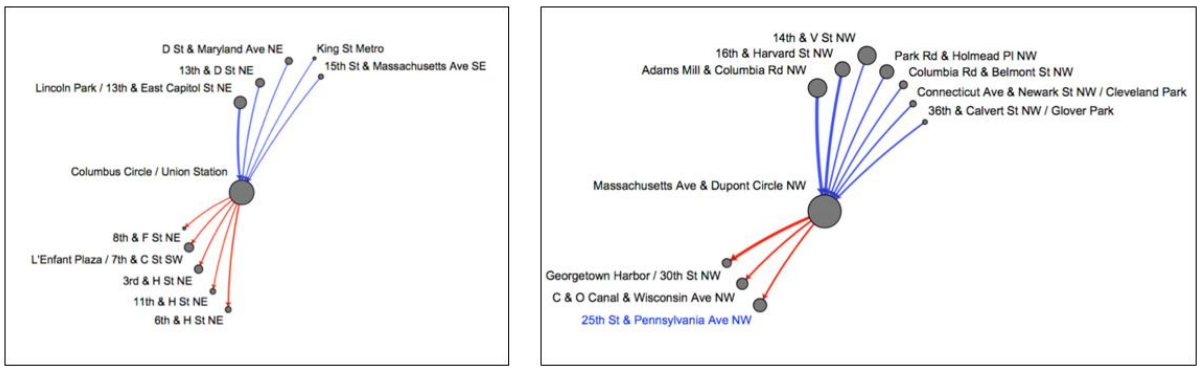
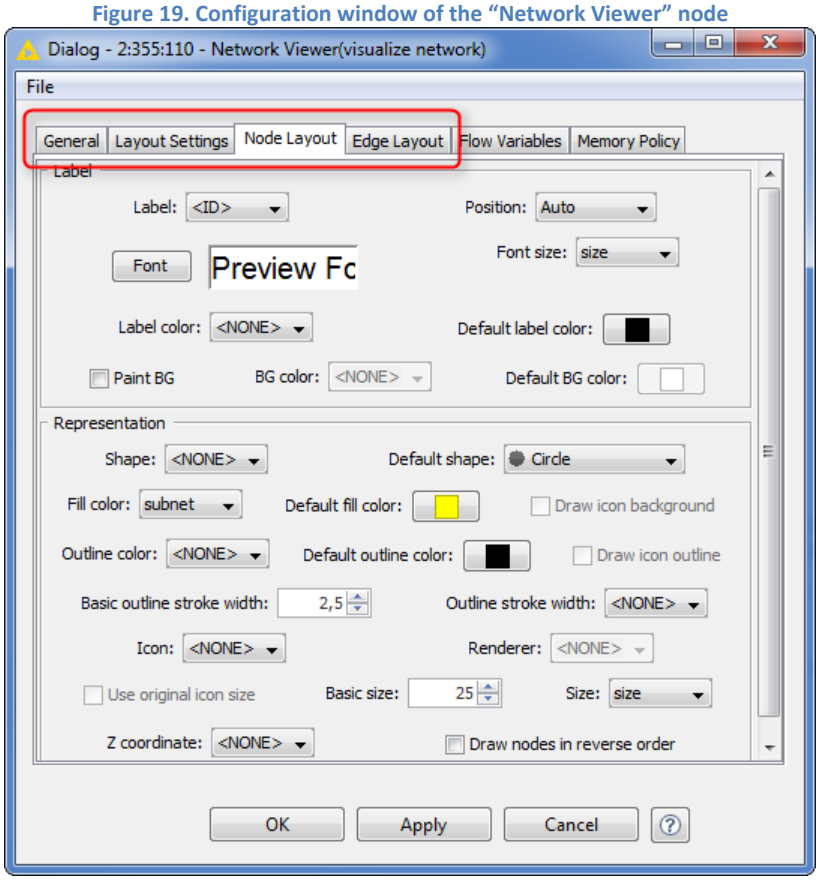


Figure 19. Configuration window of the "Network Viewer" node



A Lean Restocking Alert System

Having taken a look at data visualization, let's now return to the original goal of this study: to implement an alert system for the timely reshuffling of the bikes across stations. As an additional feature, we would also like to be able to predict the tourist wave and the registered bikers' fluctuations each day. This would help to prepare for the right number of customers each day.

We now need to enter the world of predictive analytics. Pure visualization and pure data manipulation can assist us no longer. Two workflows are built in this section, "Bikers Prediction" and "Lean Restocking Alert System", both contained in the "Predictive Analytics" workflow group.

First, we want to implement an alert signal to warn us when bikes need to be replenished (both adding and removing bikes) at a station (workflow "Lean Restocking Alert System").

Of course the easiest warning system gives the alert the moment when the number of bikes reaches zero or exceeds the number of dockings available at a station. However this method will involve idle times, which means that customers will be required to wait for free bikes or docks. Not optimal, really!

Defining the Target Variable

The system should at least mimic what humans are currently doing; i.e. when bikes are getting close to the lower or upper limit, a van is sent for restocking. This restocking action is marked in the file "station bike changes over time.table" in the data column named "Flag". Data column "Flag" supports values "No Action", "Added Bikes", and "Removed Bikes". Data column named "Shifted" tells how many bikes have been added or removed at each time. So, the system should at least be able to fire an alert signal when the "Flag" data column presents a mark "Added Bikes" or "Removed Bikes". The target variable to predict then is the data column "Flag".

But sometimes the van starts creating those idle times for customers too late, especially if we also take the time the van takes to reach the station into account. The alert system, to be truly intelligent, should predict the human reshuffling even earlier. Let's say one hour earlier. The target values are then still the values in data column "Flag", but just one hour earlier. That means that we want to insert the value of flag from the row with time 11:00am into the row with time 10:00am (and same date).

After sorting the data by descending time, a "Lag Column" node shifts the "Flag" column values down one hour in time ("Flag[-1]"). That is, if "Removed Bikes" happened at 11am in column "Flag", it is now available at 10am in column "Flag[-1]". "Flag[-1]" has become our target variable, i.e. the variable containing the alarm to predict.

Note. The "-1" in the column name "Flag[-1]" comes automatically from the "Lag Column" node and indicates that column has been moved one step down in the data table. Given the time descending sorting, then "-1" indicates the value from the future hour.

Adding New Input Features

Let's now check the other data available in the original files, i.e. the data besides "Flag" and "Shifted" that can be used as input features.

After joining the files with the data about bike restocking, "station bike changes over time.table" and "hours.csv", we have:

- Weather related features
- The number of registered and casual people that show up
- Station infos (name and maximum number of available docks)
- Calendar infos (working day, holiday, date)
- A count, as the number of bikes added and removed at each hour
- Adjusted Cumulative Sum as the total number of bikes available at a station at each hour

"Adjusted Cumulative Sum" is the data column we are interested in. It describes the total number of bikes available at a bike station at a given hour. This is the number, which, together with the "total docks available" value, will probably give the key for the alarm firing threshold. We built a more descriptive input feature combining these two numbers together in a ratio, such as:

```
Bike ratio = $Adjusted Cumulative Sum$/$total docks available$
```

This ratio is calculated in the last "Math Formula" node in the "Pre-Processing" metanode in the initial upper branch with light blue background of the "Lean Restocking Alert System" workflow (Fig. 21).

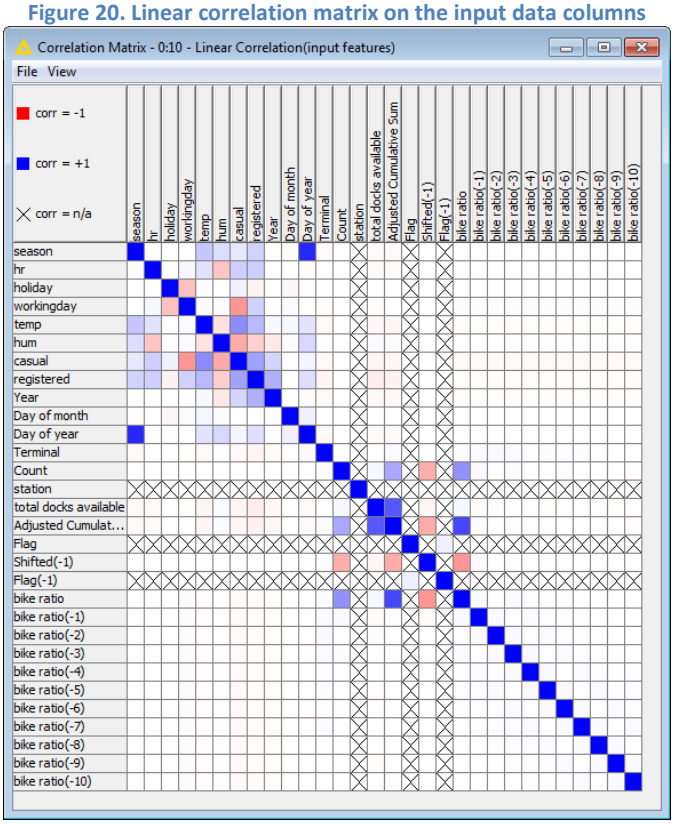
Are there more input features we might consider besides these ones? Current input features have been obtained from the original files and from web enrichment data and describe the situation at a

given hour. Would past trends help the system to estimate the restocking time more precisely? How could we produce input features about past bike usage trend? The “Lag Column” node in the central part of the workflow, with the pink background, incorporates the past 10 hours of bike ratio values, by setting the “Lag” parameter to 10 (remember: each row in the input data table represents one hour). The final input feature set, then, consists of all original data columns, plus the bike ratio, as defined above, and the past 10 hours of bike ratio values for each row.

Linear Correlation

It is often assumed that the more input features you have, the more information your machine learning algorithm will find. This has become especially common with the advent of big data algorithms. However, there are cases when new input features add irrelevant or redundant information or even worse add wrong information to the initial set of data. We have added 11 new features. Will they be useful or have we just added redundant information?

Figure 20. Linear correlation matrix on the input data columns



Let’s have a look at the linear correlation matrix produced by the “Linear Correlation” node on all available input features (Fig. 20). In general, all features are loosely correlated to each other. In particular, since correlation can only be calculated between same type data columns, correlation cannot be established between “Flag[-1]” and most other input features. However, “Flag”, and therefore “Flag[-1]”, have been generated from “Shifted” data column (i.e. the number of bikes moved from or to a station). So, let’s check the correlation between “Shifted” and the other data columns.

We see a little inverse correlation between “Shifted[-1]” and the number of bikes arriving or leaving the station (“Count”), between “Shifted[-1]” and the total number of bikes available at the station (“Adjusted Cumulative Sum”), and between “Shifted[-1]” and “bike ratio”. “bike ratio”, “Count”, and “Adjusted Cumulative Sum” are also related with each other. Correlation between “Shifted[-1]” and all other data columns is negligible. This means that these three mentioned data columns are the most informative features, but are probably redundant and only one of them should be used.

As far as other data columns are concerned, we observe some correlation between the weather and time of the year related features (humidity vs. temperature, humidity vs. season, etc…) and between the number of casual customers and weather and calendar related features (humidity, temperature, and working day flag vs. casual customers). The number of registered customers seems to be less correlated to weather and calendar features.

As usual, though, correlation tells only half of the story. Let's proceed with building the alert system and maybe learning something more about the input features from it.

Classification Algorithm

In order to predict "Flag[-1]", we can use any algorithm that allows for prediction of nominal values, like Logistic Regression, Decision Tree, Naïve Bayes, etc ... We have decided to use a simple and commonly used algorithm, the Decision Tree.

We are looking for relatively rare events ("Added/Removed Bikes" compared to "No Action"). Decision trees might underperform when classifying rare events. In order not too lose the rare events in the statistics of the algorithm, we need to present an equal number of examples for the three classes. "Equal Size Sampling" is the node that down-samples the class examples in the input data table to the same a prori probability.

After reducing the three output classes to the same count in the input data table, a "Column Filter" removes all features related to "Flag[-1]" (they would not be available in production), and columns "Count", "Adjusted Cumulative Sum", and "total docks available", since they are related to "bike ratio".

Input Feature Selection

Even after this column screening, we are still left with 29 input features. It would be optimal to use only the truly informative input features. The algorithm would then run more easily, faster, and produce results that are easier to understand.

The "Feature Elimination" metanode can be found under "Mining"/"Feature Selection"/"Meta Nodes" in the Node Repository and implements the backward feature elimination algorithm to isolate the smallest subset of the input features still with the highest performance.

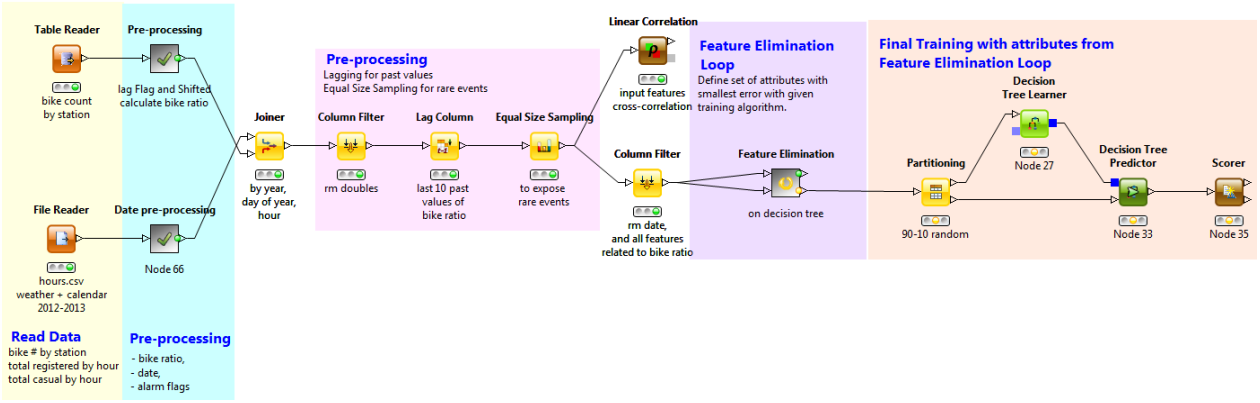
The backward feature elimination algorithm repeats the same loop many times: partitions the data, trains a selected machine learning algorithm, measures its error on a test set. At iteration 0, it executes the loop on all n input features. At iteration 1, the algorithm removes one input feature at a time and re-runs the same loop on n-1 input features; at the end of the iteration the input feature whose removal has caused the least increase in error on the test set will be removed for good. At iteration 2, the algorithm again removes one of the remaining input features at a time and re-runs the same loop on n-2 input features; again at the end of the iteration the input feature whose removal has caused the least increase in error on the test set will be removed for good. And so on, until only one input feature, the most informative one, is left in the data set. This is implemented with a "Backward Feature Elimination Start" node and "Backward Feature Elimination End" node, running a "Partitioning" node and a Learner-Predictor block as loop body.

The best error for each iteration and the corresponding input features set are then shown in the "Backward Feature Elimination Filter" node at the end of the subworkflow in the metanode. The "Backward Feature Elimination Filter" node can run in automatic or in manual (interactive) mode. In manual mode, you can select the optimal feature set you want to use. In automatic mode, you can define the error threshold for the automatic selection of the input feature subset.

We should point out here that the Learner-Predictor block inside the “Backward Feature Elimination” loop can refer to any machine learning algorithm. The default machine learning algorithm, at the creation of the metanode in your workflow, is the Naïve Bayes. However, this can be easily removed and substituted with any other machine learning pair of nodes.

The final “Lean Restocking Alert System” workflow is shown in Figure 21 below.

Figure 21. The “Lean Restocking Alert System” workflow



Results from the “Backward Feature Elimination” Filter Node

Using a decision tree, the best performances in terms of smallest error were obtained with four input features only: current “bike ratio”, “hour” of the day, “Terminal”, and “workingday” (weekend or not weekend).

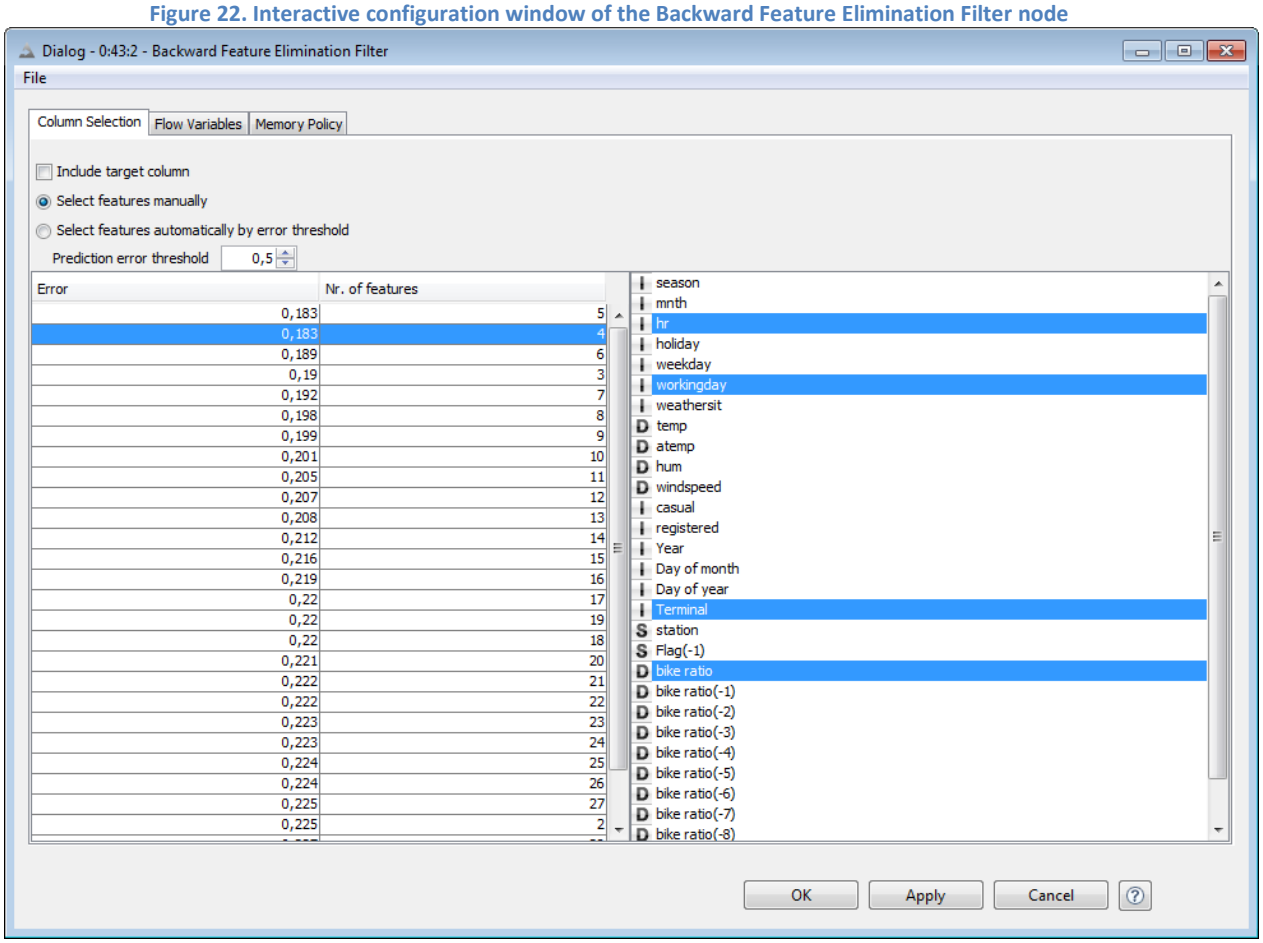
Of course, “bike ratio” is necessary to the analysis to know how many bikes are at the station with respect to the available docks. “hour” of the day also makes sense, since it is necessary to predict rush hours for registered users as well as tourism hours for casual users. “Terminal” also adds useful information since different stations have different traffic and therefore different bike restocking needs. And finally, knowing if today is a working day or a holiday helps to predict today’s traffic.

Interestingly enough, the past history for “bike ratio” as well as any weather information seems to be irrelevant. Running a decision tree on these four features gives 82% accuracy against a 77% accuracy using all 29 features (Fig. 22). If we select a threshold 0.2 for the error in the “Backward Feature Elimination Filter” node, we can train our decision tree merely with “hour”, current “bike ratio”, and “Terminal” and still get an accuracy of 80%.

Similarly, if you run a decision forest (“Tree Ensemble Learner” node), most of the time the first and second cuts are made on “bike ratio” and “Terminal”. “Terminal” and “bike ratio”, thus, seem to be the most important attributes to predict the time when bike restocking is needed. In conclusion, we can use as many attributes as we want and even use big data for it, but the algorithm performance does not improve by adding irrelevant or redundant features.

The goal of this workflow is simply to generate a restocking alarm signal. For that, past history seems to be irrelevant. However, in order to predict numbers of customers, history might play a more important role.

Figure 22. Interactive configuration window of the Backward Feature Elimination Filter node



Prediction of Total Number of Bikers by the Hour

In the hours.csv file there are two data columns, “casual” and “registered”, which contain the total number of casual and registered customers respectively using bikes in the city at each hour. Plotting these two time series using a “Line Plot” node, we can see that:

- on the full time window business has picked up nicely, given the increase in the number of customers in time, both registered and casual;
- on a monthly scale, there is a clear weekly and daily seasonality for the registered customers, while casual customers show more random peaks, usually on weekends;
- the number of casual bikers is generally lower than the number of registered bikers.

The prediction of the number of registered and casual customers using bikes throughout the city in the next hour would also help with the definition of the alert system. The goal here is to predict the number of bikers in the two categories for the next hour: it is a classic time series analysis problem.

Time Series Analysis Metanodes

Since KNIME 2.10, the “Time Series” category hosts a few new dedicated metanodes for time series analysis: “Seasonality Correction”, “Time Series Auto-Prediction Training”, and “Time Series Auto-Prediction Predictor”.

Let’s concentrate on the “registered” column as time series. After sorting the rows in ascending time, in the “Pre-Processing” metanode of the “Bikers Prediction” workflow (Fig. 24), we are ready for the time series analysis, that is:

- Seasonality Correction: x(t) = x(t) – x(t-seas_index)
- Auto-Predictive Model Training
- Auto-Predictive Model Evaluation
- Rebuilding the signal by reintroducing the seasonality pattern

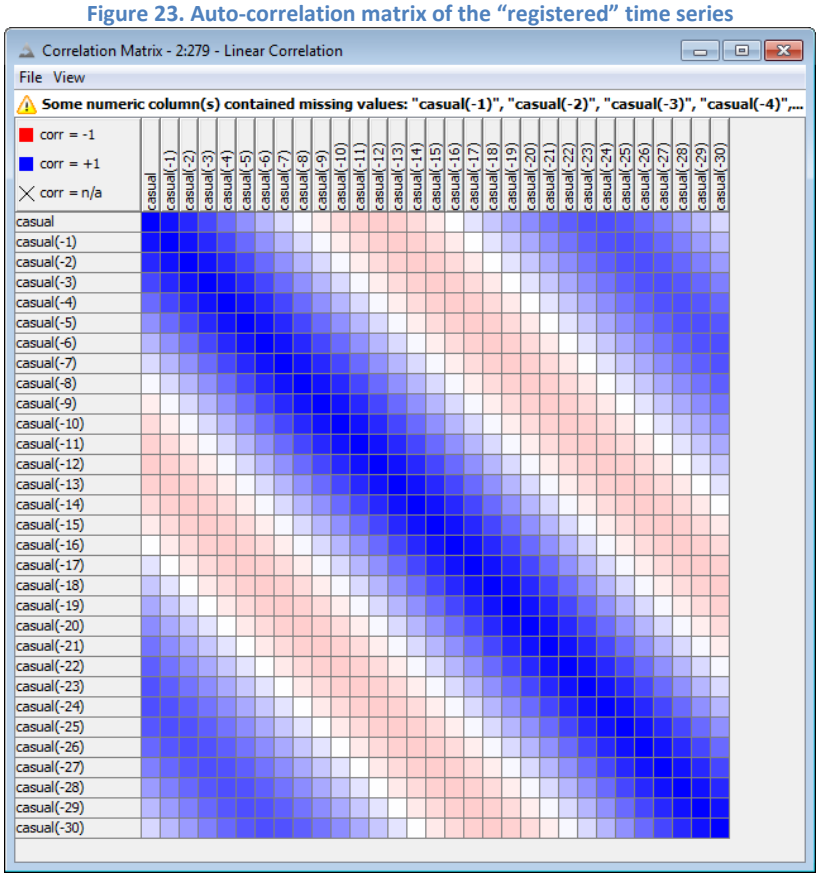
The final workflow implementing all those steps is shown in Figure 24.

Seasonality Correction

The "Seasonality Correction" metanode provides a template for seasonality correction. The "Lag Column" node provides the season template. The following "Java Snippet (simple)" node subtracts this template from the current time series.

The auto-correlation matrix, i.e. x(t) vs. x(t-i), of the "registered" time series is calculated with a "Linear Correlation" node and shows a clear 24 hour pattern (Fig. 23). A possible value then for the "Lag Interval" setting in the "Lag Column" node would be 24. In this way, the node generates a 24-hour past "registered" time series on the side of the original "registered" time series. The "Java Snippet (simple)" node subtracts the 24-hour past value from the current value, de facto removing the previous 24 hours pattern from the current one.

Figure 23. Auto-correlation matrix of the "registered" time series



Time Series Auto-Prediction Training

The "Time Series Auto-Prediction Training" metanode trains a model to predict a time series value based on its past value. In order to do that, it needs: the past values, a training set, and a model to train. Any model producing numerical prediction will suffice.

For our "registered" time series, we create 10 past values on each row with the "Lag Column" node ("Lag"=10), we build the training set using 90% of the data rows taken from the top, and finally we train a Linear Regression model.

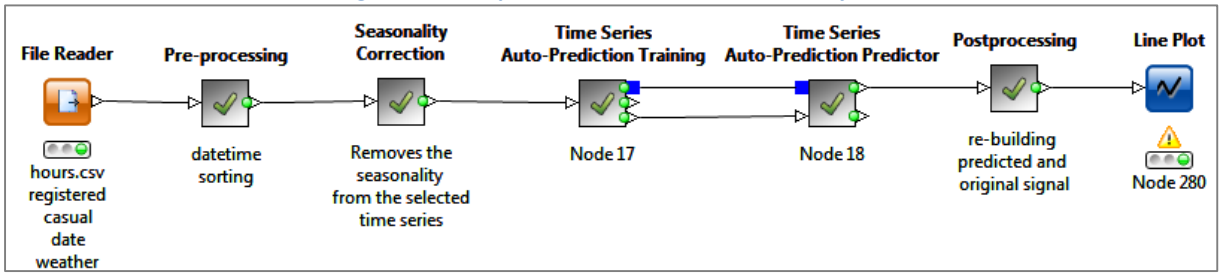
Just a remark: the option “taken from the top” in the “Partitioning” node is necessary to create the training set, because the model can only be be trained on past values.

Time Series Auto-Prediction Predictor
The “Time Series Auto-Prediction Predictor” metanode takes a model, uses it to predict future values of the time series, and calculates the prediction error.

Value prediction is performed by a Predictor node that must be consistent with the Learner node in the “Time Series Auto-Prediction Training” metanode. Prediction error is measured with a “Numeric Scorer” node, which calculates R^2, mean absolute error, mean squared error, root mean squared error, and mean signed difference.

Re-building the Signal with Seasonality Template
Finally, seasonality needs to be reintegrated into the predicted signal for realistic values. In the workflow depicted in Figure 24, this is done with two “Math Formula” nodes adding “registered[-24]” back into the “registered” time series and into the “Prediction (registered)” time series.

Figure 24. A simple Workflow for Time Series Analysis



Optimization Loop

The simple workflow in Figure 24 is based on two assumptions: there is a 24-hour seasonality pattern and 10 past values are enough to make a good prediction. How can we be sure that these are the best values for our time series prediction model?

When it comes to determining best values, the optimization loop can help. The optimization loop starts with a “Parameter Optimization Loop Start” node and ends with a “Parameter Optimization Loop End” node. The “Parameter Optimization Loop Start” node generates new flow variable values at each iteration, as specified in its configuration window. The “Parameter Optimization Loop End” node collects the objective function values from a flow variable and sorts them to find the maximum or minimum at the end of the loop.

For this project, we would like to find the best values for the seasonality index and for the number of past values used to train the model. Therefore, the “Parameter Optimization Loop Start” node needs to loop on a number of plausible values for the “Lag Interval” parameter (seasonality index) in the “Seasonality Correction” metanode and for the “Lag” parameter (number of past values) in the “Time Series Auto-Prediction Training” node. The loop body is the sequence “Seasonality Correction” – “Time Series Auto-Prediction Training” – “Time Series Auto-Prediction Predictor”, as shown in the simple workflow for time series prediction in Figure 24. At each iteration, a model is created with specific values for “Lag Interval” and “Lag” parameters.

The error measures calculated by the “Numeric Scorer” node in the “Time Series Auto-Prediction” predictor node represent the function to minimize (the goal of this loop is to get the best model, and best here means with smallest error).

The configuration windows of the "Parameter Optimization Loop Start" node and "Parameter Optimization Loop End" node each are shown below. Flow variable "lag_par" produces the number of past values at each iteration as 1, 2, … 20; while flow variable "seas_par" produces the seasonality index to be used at each iteration, as 1, 2, … 30. The "Parameter Optimization Loop End" node selects the flow variable named "root mean square deviation", as produced by the "Numeric Scorer" node and containing the RMSE prediction error.

Figure 25. Configuration window of the Parameter Optimization Loop Start node

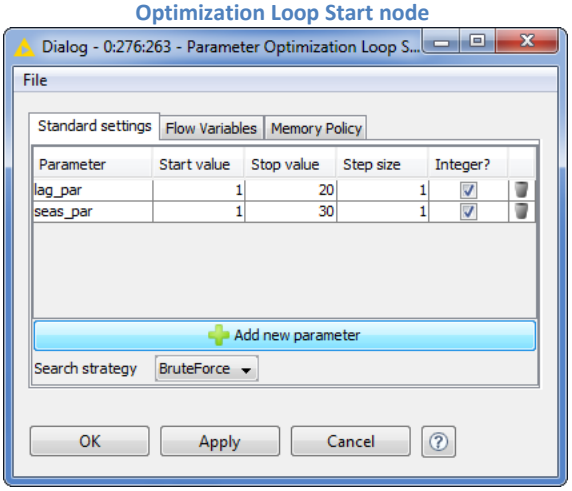
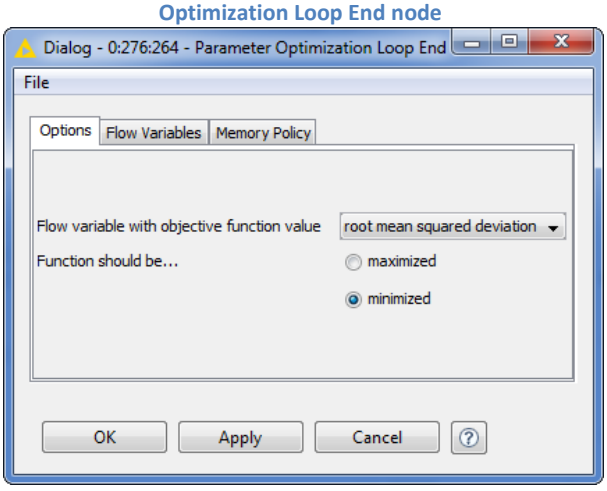


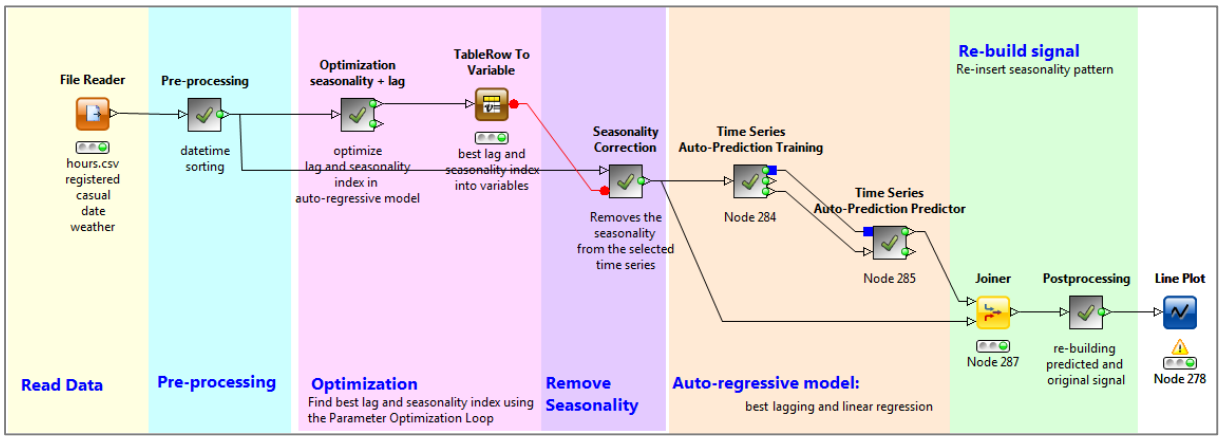
Figure 26. Configuration window of the Parameter Optimization Loop End node



The optimization procedure can follow two search strategies: "BruteForce" and "HillClimbing". "BruteForce" runs through all possible combinations of the loop parameters, while "HillClimbing" starts from a random subset of such combinations and explores only the direct neighbor combinations. The second strategy is of course faster, even though sometimes less precise.

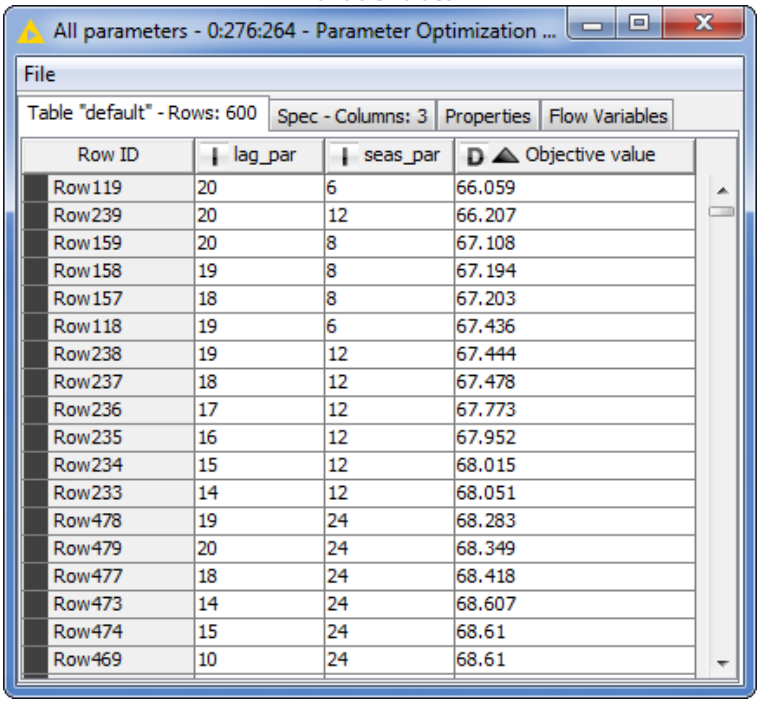
We try all possible parameters combinations (brute force optimization loop) and select the combination producing the lowest Root Mean Square Error. This combination of best parameters is then applied to the final time series prediction metanodes (Fig. 27).

Figure 27. Final workflow for time series analysis including an optimization loop in the "Optimization Seasonality + lag" metanode



The output of the "Parameter Optimization Loop End" node shows that the best results in terms of prediction error on the test set are obtained with seasonality index 6, 12, 8, and 24. The lowest error was actually obtained with seasonality index 6 and 20 steps backwards in time.

Figure 28. Results of the optimization loop on "lag_par" (number of past values) and "sea_par" (seasonality index) flow variable values



Row ID	lag_par	seas_par	Objective value
Row119	20	6	66.059
Row239	20	12	66.207
Row159	20	8	67.108
Row158	19	8	67.194
Row157	18	8	67.203
Row118	19	6	67.436
Row238	19	12	67.444
Row237	18	12	67.478
Row236	17	12	67.773
Row235	16	12	67.952
Row234	15	12	68.015
Row233	14	12	68.051
Row478	19	24	68.283
Row479	20	24	68.349
Row477	18	24	68.418
Row473	14	24	68.607
Row474	15	24	68.61
Row469	10	24	68.61

Using the "casual" time series as prediction object leads to 1 as best seasonality index and 20 as best number of past time series values. 1 hour as seasonality index means that we are only using the previous value to predict the average value of the next one: no seasonality really.

Conclusions

In the project described in this whitepaper we have tackled an Internet of Things application in the form of a bike sharing business. The goal was to add some intelligence to the bike restocking part of the business to fire an alarm signal well before a bike station exceeds the number of docks available or runs out of bikes.

The project consisted of three phases, as it is often the case: data preparation, data visualization, and predictive analytics.

Data preparation collected a number of files describing the bike rentals, the bike rides, and the bike stations spread around Washington DC. It also took care of enriching the original data set with weather, calendar, and traffic information acquired mainly via Google API RESTful services. Data were also transformed (mainly through aggregations), to reduce the list of bike rides to the net number of bikes available at each station at each hour.

In terms of visualization, experiments were carried out with a number of different techniques: bike stations were localized on a Washington DC map using the KNIME Open Street Map (OSM) Integration; bike routes in Washington DC were displayed using the ggplot R library through the KNIME R Integration extension; and, finally, bike routes were represented as edges of a network graph using the KNIME Network Analytics extension. All these different visualization techniques show a different aspect of the data and of the KNIME open architecture, which makes integration of data and tools extremely easy.

Predictive analytics has two goals: to define an alarm system for the timely restocking of the bike stations and predict the number of customers using bikes in the city at every hour. Besides the classical machine learning techniques used in both workflows, data mining and time series auto-regression

respectively, some additional optimization techniques have been adopted. A “Feature Elimination” loop builds the leanest alarm system, i.e. the system using the minimum number of necessary input features. An optimization loop defines the best values for the seasonality index and the number of necessary past values to predict the current number of customers.

This focused application for the Internet of things also has the capacity to provide a wider scope of application in that the techniques used – data cleansing, data enrichment, data visualization and machine learning – will be the series of steps used to transform any machine data (IOT or otherwise) into actionable insight. We hope that this white paper provides a foundation for further experiments and research into using IOT data for practical application.

Data and workflows are available for downloads at http://www.knime.com/white-papers#IoT , while the KNIME open source platform is downloadable from the KNIME site at http://www.knime.org/knime-analytics-platform-sdk-download .